МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №3
по курсу «Программирование графических процессоров»

Классификация и кластеризация изображений на GPU. Метод максимального правдоподобия (вариант 1)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы — научиться использовать GPU для классификации и кластеризации изображений: разобраться с константной памятью и одномерной сетью потоков.

Формат изображений такой же, как в лабораторной работе №2: бинарный файл, первые 8 байт — ширина w и высота h (целые, little-endian), дальше пиксели построчно, каждый по 4 байта: r, g, b, a. Ограничение: $w \cdot h \le 4 \cdot 10^8$. Ввод (stdin): на первой строке путь к исходному изображению, на второй — путь к конечному. Дальше число nc — количество классов ($2 \le nc \le 32$), затем nc строк с описанием классов: в начале j-й строки число np_j — количество пикселей в выборке ($2 \le np_j \le 2^{19}$), за ним np_j пар чисел — координаты пикселей выборки.

Мой вариант — №1, метод максимального правдоподобия. Для каждого класса j по выборке оцениваются вектор средних $avg_j = (1/np_j) \cdot \Sigma\ ps_i$ и ковариационная матрица $cov_j = (1/(np_j-1)) \cdot \Sigma\ (ps_i - avg_j) \cdot (ps_i - avg_j)^T$, где ps_i — i-й пиксель j-й выборки, вектор (r, g, b). Пиксель p относится к классу с максимальным правдоподобием: $pc_j = \arg\max_j\ [-(p - avg_j)^T \cdot cov_j^{-1} \cdot (p - avg_j) - \log|\det(cov_j)|]$. Если пиксель можно отнести к нескольким классам, выбирается класс с наименьшим номером. В результирующем изображении RGB-каналы не меняются, а на месте альфа-канала записывается номер класса, к которому отнесён соответствующий пиксель.

Программное и аппаратное обеспечение

Код я писал, отлаживал и компилировал на виртуальной машине с GPU, на ней же проводил все замеры времени и готовил картинки для отчёта.

Графический процессор — NVIDIA GeForce GTX 1650: compute capability 7.5, 14 мультипроцессоров, глобальная память 4 086 169 600 байт (примерно 3,8 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, максимум 1024 нити на блок, максимальная сетка (2 147 483 647, 65 535, 65 535), размер варпа 32.

Из софта: Ubuntu 24.04.1 LTS (ядро 6.8.0-84-generic), драйвер NVIDIA 550.144.03, CUDA 12.4, компилятор nvcc 12.0 (V12.0.140), g++ 13.3.0. Тестовые картинки для отчёта, бенчмарк и конвертацию бинарных картинок в PNG запускал скриптами на этой же машине (чистый Python, без сторонних пакетов).

Метод решения

Сначала читаю из stdin две строки — пути к исходному и конечному файлам, потом число классов nc и описания классов. Исходный файл открывается в бинарном режиме, заголовок читаю побайтно и собираю little-endian числа. Дальше валидация: $w, h \ge 1$, $w \cdot h \le 4 \cdot 10^8$, nc от 1 до 32, координаты пикселей выборок — в границах изображения. Пиксели читаются в массив на хосте (malloc).

Статистики классов считаю на хосте в double за один проход по каждой выборке: накапливаю суммы каналов и суммы их попарных произведений, из них вектор средних $avg_j = sum/np_j$ и ковариационная матрица $cov_j = (S_2 - np_j \cdot avg_j \cdot avg_j^T)/(np_j - 1)$ — это то же самое, что $\Sigma(ps_i - avg_j)(ps_i - avg_j)^T$, но одним проходом и без хранения координат. Дальше для каждого класса — определитель det(cov_j), log|det| и обратная матрица 3×3 через сопряжённую (алгебраические дополнения, делённые на определитель). Классификация пикселя зависит только от его RGB, поэтому класс полностью задают тройка avg_j, обратная матрица и log|det|: они копируются в константную память устройства (cudaMemcpyToSymbol) — массивы avg (nc×3), inv (nc×9) и logdet (nc), при nc

= 32 это около 3,4 КБ из 64 КБ. Константная память кэшируется, и когда все нити варпа читают один и тот же адрес, чтение работает как широковещательное — для параметров классов, общих всем пикселям, это самый удачный вид памяти.

Ядро работает с одномерной сетью потоков (как требует лабораторная), конфигурация фиксированная: grid(224), block(256), всего 57 344 нити — от размера картинки не зависит. Каждая нить ходит по пикселям одномерным grid-stride циклом с шагом gridDim.x·blockDim.x. Для своего пикселя нить в цикле по классам считает $d = p - avg_j$, квадратичную форму $q = d^T \cdot inv_j \cdot d$ (матрично-векторное произведение и скалярное) и оценку правдоподобия $score_j = -q - logdet_j$; выигрывает класс с максимальным score, при точном равенстве — меньший номер (сравнение строгое). В альфа-канал пикселя пишется номер класса, RGB не трогается.

Классификация пикселя зависит только от него самого, поэтому ядро работает in-place по одному буферу: на предельном наборе это 1,6 ГБ из 3,8 ГБ глобальной памяти (отдельный буфер под результат занял бы ещё столько же — впритык). Буфер выделяется cudaMalloc'ом и заполняется cudaMemcpy'ом с хоста, после запуска ядра и синхронизации результат копируется обратно и пишется в выходной файл: заголовок w, h (little-endian) и все пиксели. Все операции с файлами, malloc и все вызовы CUDA проверяются — при любой ошибке печатается «ERROR: …» в stderr, и программа завершается с кодом 0. При написании ориентировался на официальный CUDA C Programming Guide (docs.nvidia.com).

Описание программы

Программа — один файл lab3.cu.

Ядро classifyKernel(uchar4 *img, long long n): одномерный grid-stride цикл по пикселям, для каждого вызывается classifyPixel и её результат — номер класса — пишется в альфа-канал. classifyPixel(uchar4 p) — __device__-функция: цикл по классам, чтение avg_j, inv_j и logdet_j из константной памяти, квадратичная форма $d^T \cdot inv_j \cdot d$, оценка $-q - logdet_j$ и выбор максимума со строгим сравнением (при равенстве остаётся класс с меньшим номером).

Ещё три маленькие вспомогательные функции: fail — печать сообщения об ошибке в stderr и выход, readPath — чтение пути из stdin со срезанием перевода строки, writeLE32 — запись 4-байтового little-endian числа в файл.

main() читает пути, открывает и читает исходный файл, валидирует размеры, читает nc и описания классов — попутно накапливая суммы каналов и произведений, — считает статистики, обратные матрицы и log|det|, копирует их в константную память, выделяет буфер на устройстве, копирует туда изображение, запускает ядро, копирует результат обратно и пишет выходной файл — и всё это с проверкой ошибок.

Типы: uchar4 — пиксель (r, g, b, a по байту), double — все статистики и вычисления в ядре, long long — размеры и индексы (индекс пикселя на предельном наборе доходит до $4 \cdot 10^8$). Рабочая конфигурация — grid(224), block(256), фиксированная; остальные конфигурации из таблицы ниже — только для замеров.

Результаты

Время ядра мерял событиями CUDA, в таблице — среднее по 10 запускам (перед замерами — ещё 3 прогревочных запуска). Конфигурации записаны как <<<блоки, нити>>>; сеть и блок одномерные, как требует лабораторная. Взял три входных набора: 512×512 (маленький), 4096×4096 (средний) и 10000×10000 (предельный, тут $w \cdot h = 10^8$). Результат каждой конфигурации сверялся с CPU-эталоном (39 проверок, все совпадения точные). Для сравнения — CPU-версия (один поток, та же математика в double, компилируется тем же nvcc в том же бинарнике).

Конфигурация (блоки × нити)	Всего нитей	Малый 512×512, мкс	Средний 4096×4096, мс	Предельный 10000×10000, мс
<<<1, 32>>>	32	16016,4	885,61	5316,27
<<<1, 64>>>	64	7777,6	535,35	3204,84
<<<1, 128>>>	128	6817,1	436,53	2622,39
<<<1, 256>>>	256	6813,4	435,65	2617,18
<<<1, 512>>>	512	6814,4	435,62	2632,62
<<<1, 1024>>>	1 024	6814,3	437,01	2638,00
<<<4, 256>>>	1 024	1710,2	109,85	659,89
<<<16, 256>>>	4 096	858,1	54,92	329,91
<<<64, 256>>>	16 384	538,2	34,33	206,13
<<<224, 256>>>	57 344	499,1	31,39	188,53
<<<256, 256>>>	65 536	510,8	32,59	195,79
<<<1024, 256>>>	262 144	498,0	31,73	190,63
<<<1024, 1024>>>	1 048 576	513,6	31,76	191,15
CPU (1 поток)	1 поток	17737,7	1119,41	6629,64

Картина такая. Пока блоков мало, время обратно пропорционально их числу, а после ~16 384 нитей выходит на плато (на маленьком наборе 498–514 мкс). Лучшая конфигурация — <<<224, 256>>>: 499,1 мкс на маленьком наборе, 31,39 мс на среднем и 188,53 мс на предельном; она же используется в программе (на маленьком наборе чуть быстрее <<<1024, 256>>> — 498,0 мкс против 499,1 мкс, но на предельном выигрывает <<<224, 256>>>). Конфигурация с 1 048 576 нитями на маленьком наборе уже хуже (513,6 мкс) — расходы на запуск слишком большой сетки. Сеть из одного блока — это один мультипроцессор из четырнадцати: одномерные конфигурации <<<1, 128>>>…<<<1, 1024>>> дают практически одно и то же время (2,62–2,64 с на предельном наборе), его определяет латентность памяти одного мультипроцессора, а не пропускная способность. С 32 нитями GPU уже не медленнее CPU (5,32 с против 6,63 с на предельном наборе), при 1 024 нитях в конфигурации <<<4, 256>>> — быстрее в десять раз (0,66 с), а на насыщении — в 35 раз.

Ускорение против одноядерного CPU практически одинаковое на всех трёх наборах — около 35 раз (35,5× на маленьком, 35,7× на среднем и 35,2× на предельном: 188,53 мс против 6 629,64 мс). На предельном наборе эффективный трафик — 4 байта чтения + 4 байта записи на пиксель (ядро работает in-place, каждый пиксель читается и пишется один раз), то есть 800 МБ за 188,53 мс — около 4,2 ГБ/с при пиковой пропускной способности

GTX 1650 порядка 128 ГБ/с. Значит, ядро упирается не в память, а в вычисления: для каждого пикселя и каждого из 5 классов считаются разности, квадратичная форма 3×3 и сравнение оценок — всё в double, а двойная точность на GeForce выполняется с многократно меньшей скоростью, чем одинарная.

Тестовую картинку для отчёта генерировал скриптом на Python: 512×512, пять областей — фон, эллипс, прямоугольник, треугольник и линия, плюс детерминированный шум ±3 на каждый канал (без шума сплошные цвета дали бы вырожденные ковариационные матрицы). Выборки классов — по 64 точки из каждой области. Программа классифицировала картинку без единой ошибки: все 100% пикселей каждой из пяти областей отнесены к своему классу (проверено сравнением с эталонной разметкой областей). На рисунке 2 номер класса из альфа-канала показан цветами палитры.

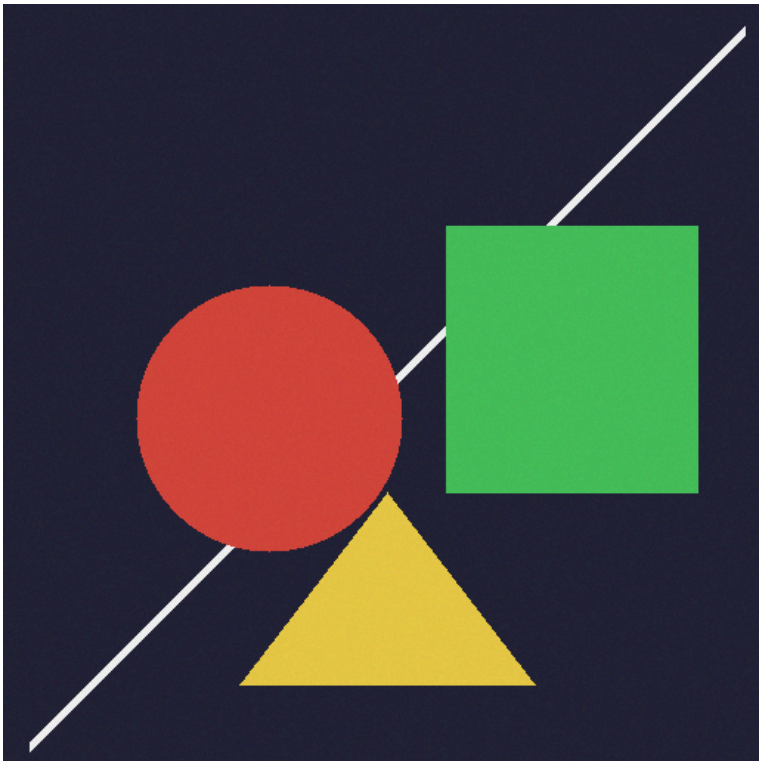


Рисунок 1 — Исходное изображение (512×512)



Рисунок 2 — Результат: классификация методом максимального правдоподобия (512×512, номер класса показан цветом)

Выводы

Метод максимального правдоподобия — статистический метод классификации с учителем: каждый класс моделируется трёхмерным нормальным распределением, параметры которого (вектор средних и ковариационная матрица) оцениваются по обучающей выборке, и пиксель относится к классу с максимальной плотностью вероятности. Применяется в дистанционном зондировании (классификация мультиспектральных спутниковых снимков), сегментации медицинских изображений и компьютерном зрении. В отличие от метода минимального расстояния, критерий с квадратичной формой и $-\log|\det|$ учитывает и форму класса: вытянутый класс штрафуется вдоль «узких» направлений, а компактный — нет, поэтому метод различает классы с одинаковыми средними, но разным разбросом.

Полученные результаты объясняются так. Производительность растёт с числом нитей лишь до насыщения GPU — примерно 16 384 нити, когда блоков хватает на все 14 мультипроцессоров, — после чего выходит на плато. Ускорение против одноядерной CPU-версии — около 35 раз на всех трёх наборах. Ядро не выходит на пик пропускной способности памяти (4,2 ГБ/с из ~128 ГБ/с): оно ограничено вычислениями в двойной точности — на каждый пиксель и каждый класс считаются квадратичные формы 3×3. Показательно, что даже конфигурация с 32 нитями обгоняет CPU: в отличие от лабораторной №2, на пиксель приходится заметно больше арифметики, и даже одному варпу на одном мультипроцессоре есть чем загрузить вычислительный конвейер. Возможные оптимизации — одинарная точность (float) для квадратичных форм при статистиках, предвычисленных в double, и развёртка цикла по классам при фиксированном

nc. Конфигурация запуска важна: при одном блоке работает один мультипроцессор из четырнадцати, поэтому при малом числе блоков GPU недозагружен.